Łukasz Kolber

Aplikacja Web dla systemu Senswave

Wprowadzenie **2**
Funkcje **2**
Źródła Danych 2
Domy 2
Urządzenia 3
Dynamiczne Aktualizacje 3
Użytkownicy 3
Model Danych **4**
Źródła Danych 4
Domy 5
Dynamiczne Aktualizacje 5
Urządzenia 6
Użytkownicy 7
Frontend **8**
Schemat Organizacji Treści / Nawigacja 8
Standardy WACG 8
Walidacja Danych 9
Backend **10**
Kontekst Systemu 10
Architektura wewnętrzna - Kontenery 10
Komunikacja i Bezpieczeństwo 11
Mechanizmy zabezpieczeń 11
Interfejs REST API 11
Łączność w czasie rzeczywistym (SignalR) 12

Wprowadzenie

Senswave to system realizujący funkcje smart home, przeznaczony dla urządzeń w pełni tworzonych przez użytkownika. Rozwiązanie umożliwia definiowanie funkcji urządzenia, na podstawie których generowane są kontrolki. Mogą być one dowolnie rozmieszczone przez użytkownika, co oznacza, że to on odpowiada za czytelność interfejsu.

Po utworzeniu urządzenia istnieje możliwość jego udostępnienia innym użytkownikom zarejestrowanym w aplikacji.

Funkcje

System składa się z kilku niezależnych modułów, które mogą być wykorzystywane również w innych systemach. Do głównych modułów należą: użytkownicy, źródła danych, domy, urządzenia oraz dynamiczne aktualizacje.

Każdy z nich posiada własny zestaw funkcji, które mogą być ze sobą powiązane i współdziałać w ramach całego systemu.

Źródła Danych

Są to zewnętrzne serwery MQTT, z którymi Senswave łączy się na żądanie użytkownika. Służą do przekazywania danych do i z urządzeń.

Funkcje modułu:

- zarządzanie źródłami danych użytkownika: (CRD)
- zarządzanie połączeniem systemu ze źródłem danych:
    - rozpoczynanie sesji z serwerem,
    - kończenie sesji,
    - pobieranie stanu połączenia,
    - utrzymywanie sesji po wylogowaniu użytkownika,
- przekazywanie ruchu do i z urządzeń.

Domy

Dom stanowi podstawową jednostkę systemu, umożliwiającą agregowanie urządzeń oraz ich udostępnianie innym użytkownikom do wspólnego korzystania.

Funkcje modułu:

- zarządzanie domami (CRUD)
- udostępnianie domów innym użytkownikom,
- podział na pokoje, (CRUD)
- powiązanie ze źródłami danych.

Urządzenia

Celem modułu jest tworzenie wirtualnego odwzorowania urządzeń oraz udostępnianie ich funkcji w ramach systemu. Jest to najbardziej rozbudowany element Senswave, odpowiadający za logikę działania urządzeń oraz sposób ich prezentacji w interfejsie użytkownika.

Moduł umożliwia pełną konfigurację zarówno funkcjonalności urządzeń, jak i ich wizualnej reprezentacji, co pozwala na dostosowanie systemu do indywidualnych potrzeb użytkownika.

Funkcje modułu:

- zarządzanie urządzeniami oraz ich kafelkami na ekranie domu (CRUD),
- zarządzanie operacjami urządzeń (CRD),
- zarządzanie kontrolki urządzenia (CRUD),
- zarządzanie konfigurowalnymi ekranami (CRUD).
    - wiązanie z kontrolkami
- wiązanie z źródłem danych domu

Dynamiczne Aktualizacje

Zadaniem modułu jest przechwytywanie zdarzeń zachodzących w systemie oraz informowanie o nich użytkownika aktualnie korzystającego z aplikacji.

Moduł odpowiada za bieżącą komunikację zmian, dzięki czemu użytkownik ma stały wgląd w stan urządzeń i interfejsu.

Funkcje modułu:

- powiadamianie o zmianach na ekranie urządzenia,
- powiadamianie o zmianach stanu na ekranach konfigurowalnych z kontrolkami.

Użytkownicy

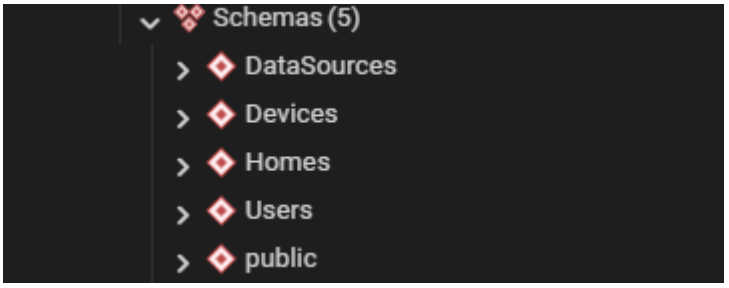
Jest to moduł odpowiedzialny za zarządzanie danymi użytkownika. Stanowi standardowy element systemu, realizujący podstawowe funkcje związane z uwierzytelnianiem i obsługą kont.

Funkcje modułu:

- logowanie:
    - przy użyciu adresu e-mail i hasła,
    - z wykorzystaniem konta Google,
- obsługa tokenów typu Bearer oraz ich weryfikacja,
- rejestracja użytkownika,
- usuwanie konta.

Model Danych

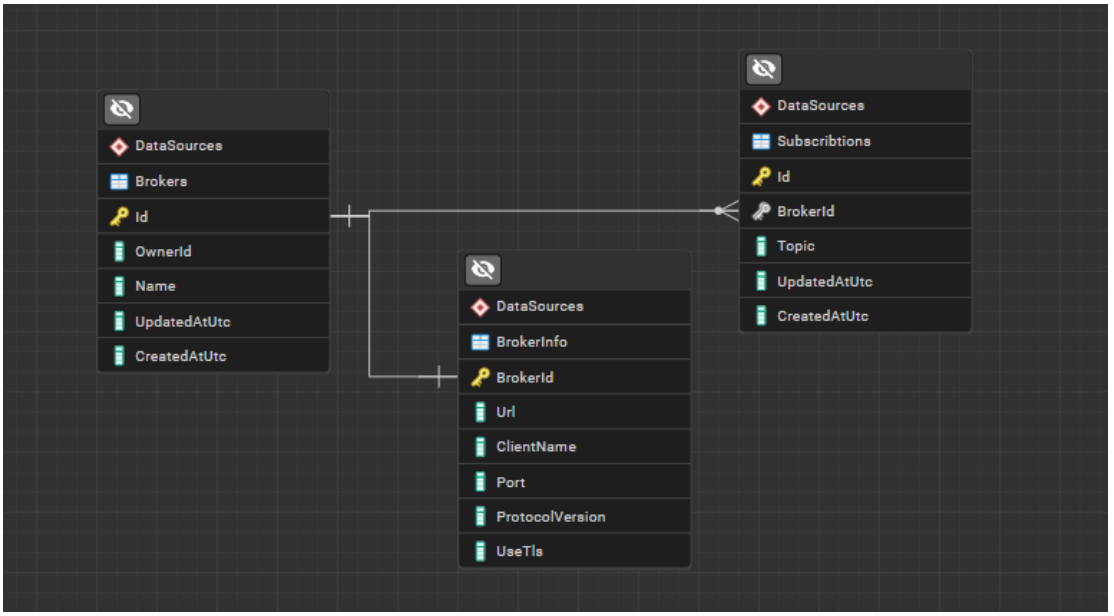
Wszystkie dane znajdują się w jednej bazie danych ale aby wymusić modularność rozwiązania zastosowany został podział na schematy (Rys. 1).



Rys.1 Zrzut ekranu dostępnych schematów w bazie danych z programu PGAdmin

Poniżej zostaną przedstawione zrzuty ekranu schematów w bazie danych per Moduł

Źródła Danych

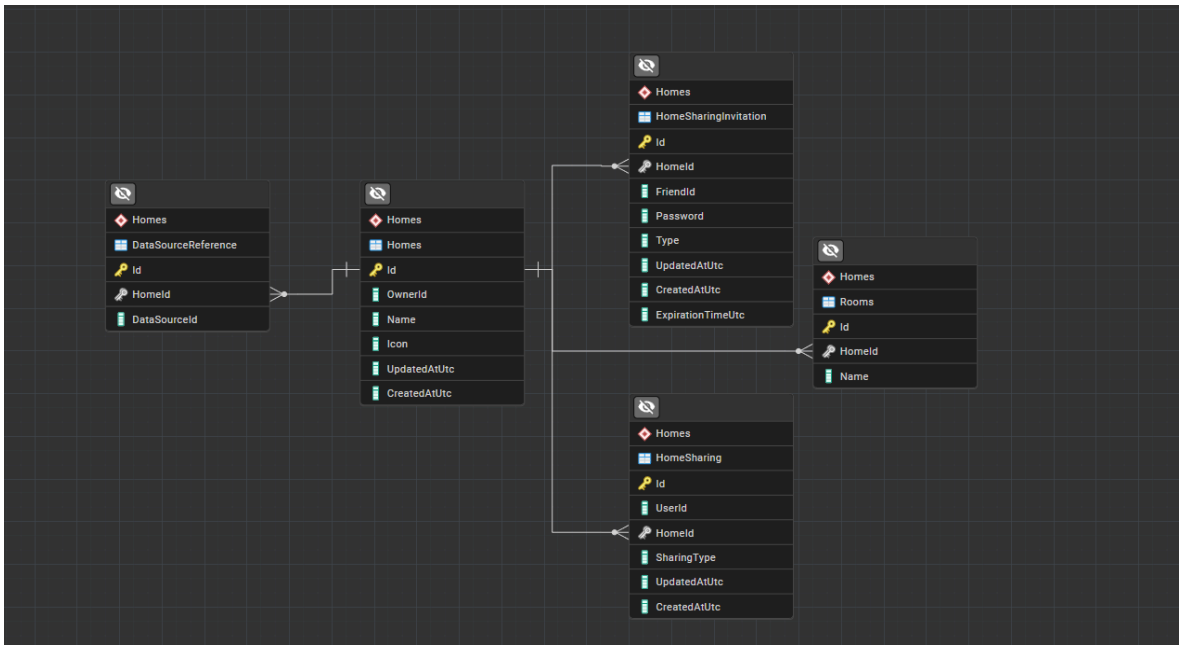


Rys.2 Zrzut ekranu tabel dla modułu źródeł danych z programu PGAdmin

(Rys. 2) Tabele:

- **Broker** – główna jednostka organizacyjna reprezentująca źródło danych,
- **BrokerInfo** – szczegóły połączenia z brokerem,
- **Subscriptions** – dynamicznie tworzone tematy (topic) do subskrypcji.

Domy



Rys.3 Zrzut ekranu tabel dla modułu domów z programu PGAdmin

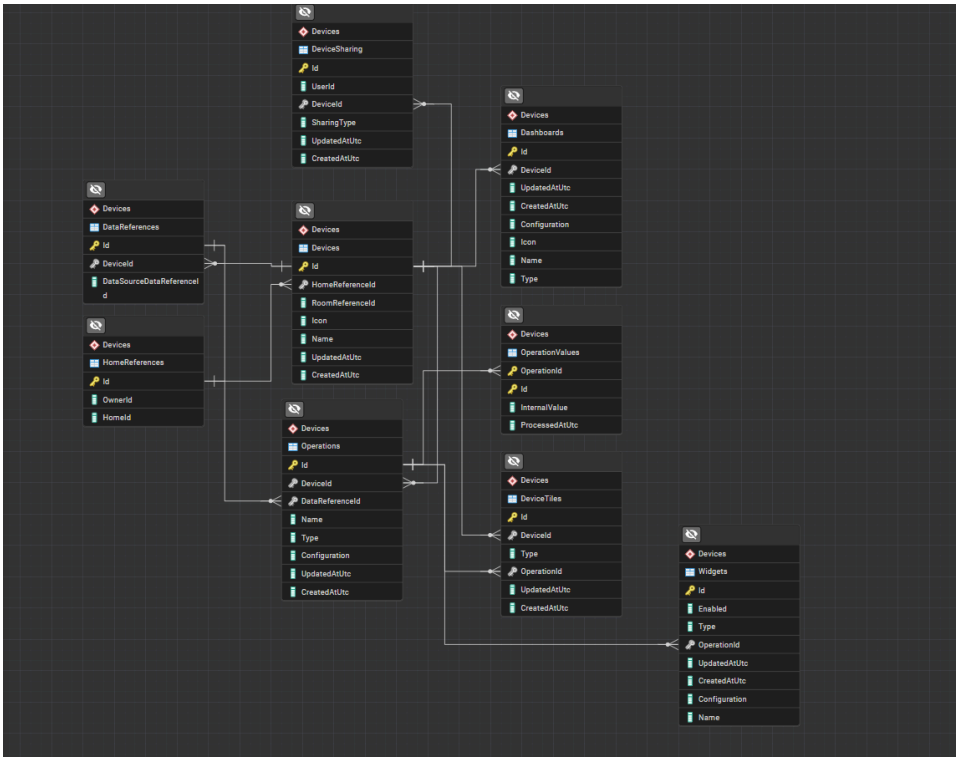
(Rys. 3) Tabele:

- **DataSourceReference** – tabela powiązań ze źródłem danych, zapewniająca spójność transakcyjną (1 dom = 1 źródło danych),
- **Home** – główny obiekt agregujący w module,
- **HomeInvitations** – zaproszenia do domu wraz z kodem potwierdzającym,
- **HomeSharing** – lista użytkowników zaproszonych do domu wraz z nadanymi uprawnieniami,
- **Rooms** – pokoje przypisane do domu.

Dynamiczne Aktualizacje

Moduł nie posiada modelu danych.

Urządzenia

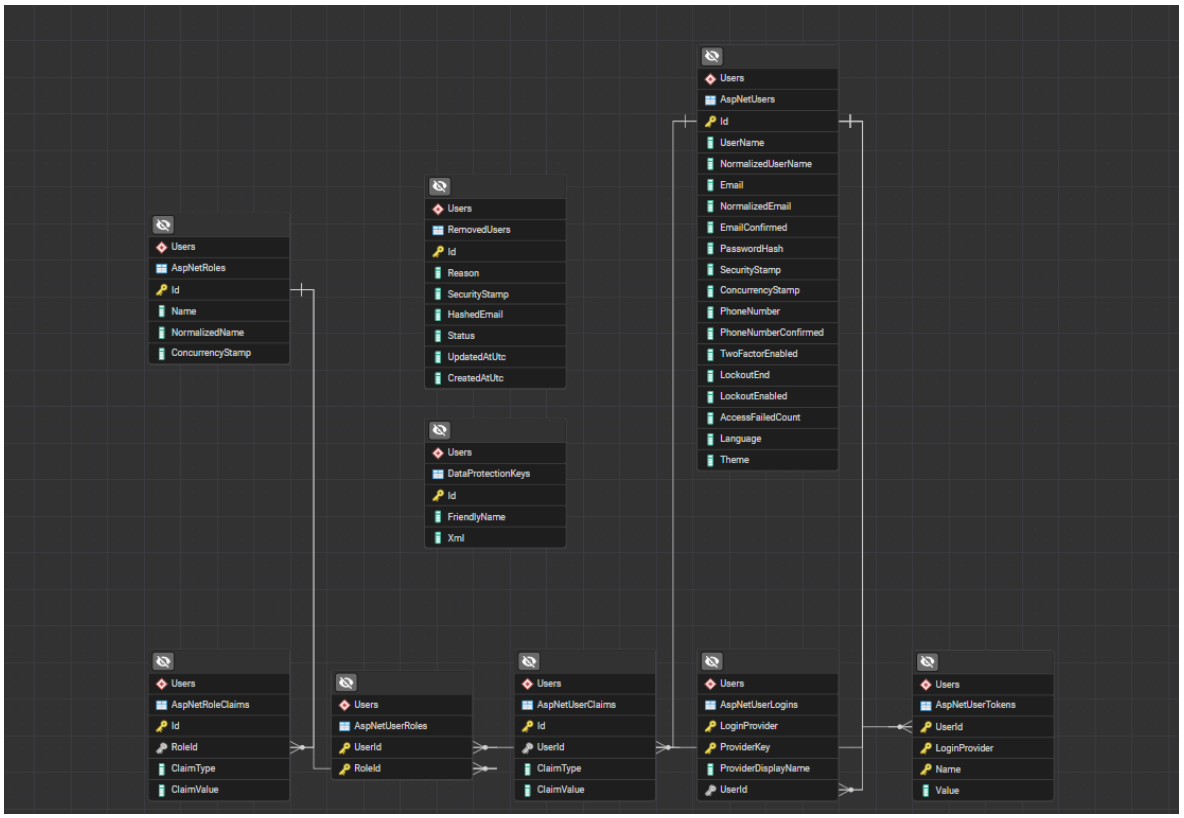


Rys.4 Zrzut ekranu tabel dla modułu urządzeń z programu PGAdmin

(Rys. 4) Tabele:

- **HomeReferences** – zarządza przypisaniem domów do ich właścicieli,
- **DataReferences** – definiuje źródła danych i zapewnia ich spójność w systemie,
- **Devices** – główny rejestr urządzeń przypisanych do domów i pokoi,
- **DeviceSharing** – zarządza uprawnieniami i udostępnianiem urządzeń użytkownikom,
- **Operations** – definiuje dostępne akcje i funkcje dla każdego urządzenia,
- **OperationValues** – przechowuje historię oraz bieżące wartości wykonanych operacji,
- **Dashboards** – przechowuje układ konfigurowalnego widoku
- **DeviceTiles** – mapuje operacje urządzeń na konkretne kafle
- **Widgets** – zarządza detalami kontrolek sterujących

Użytkownicy



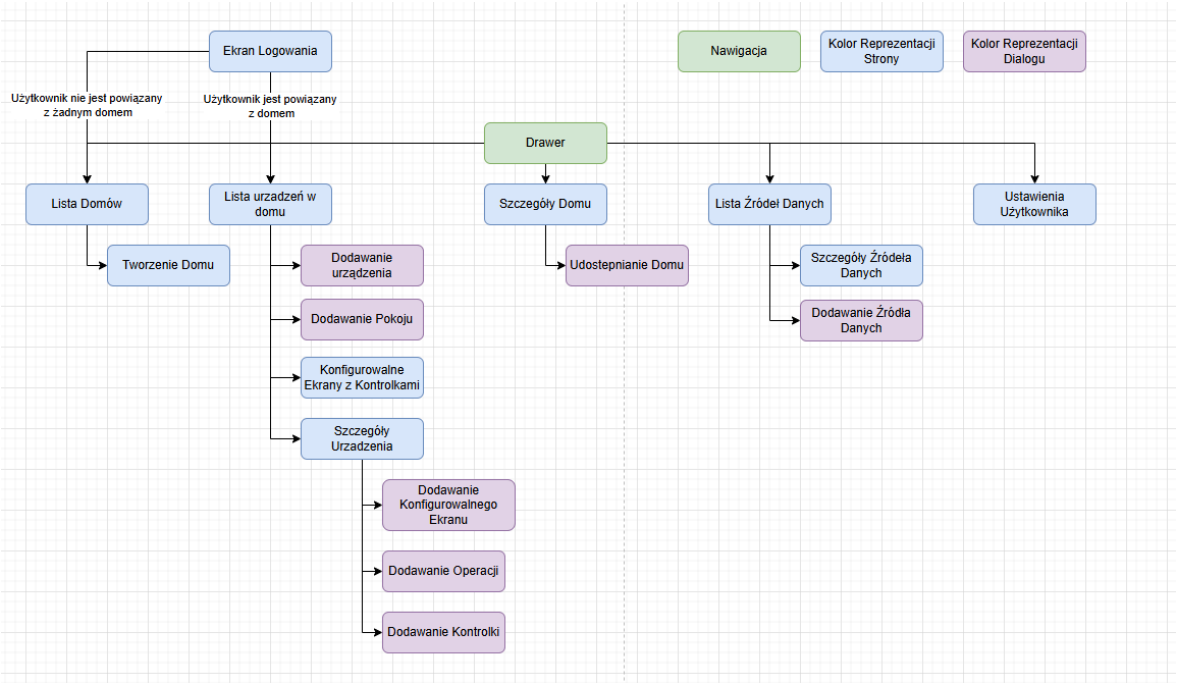
Rys.6 Zrzut ekranu tabel dla modułu użytkowników z programu PGAdmin

Tabele bazują na gotowej implementacji AspNetCore Identity. Więcej szczegółów można znaleźć w dokumentacji microsoft. Jedyna dodatkowa tabela to lista usuniętych użytkowników.

Frontend

Aplikacja została napisana w technologii Blazor WASM w jezyku .NET z elementami JS.

Schemat Organizacji Treści / Nawigacja



Rys.7 Schemat nawigacji w aplikacji

Nawigacja w aplikacji jest realizowana przez drawer który ma 3 różne stany w zależności czy używamy tabletu, telefonu czy komputera. Podstawowym elementem nawigacji jest stroną każda strona zawiera Brad Rumby pozwalające na prze nawigowanie poziom wyżej jeżeli trzeba oraz informuje gdzie użytkownik się znajduje. Liczba zagnieżdżenia strona została zredukowana dzięki zastosowaniu dialogów które pozwalają zastąpić proste strony na formularze.

Standardy WACG

1. Postrzegalne (Perceivable)

- Aplikacja oferuje trzy tryby kolorów: light, dark oraz high contrast, co poprawia widoczność treści dla użytkowników z różnymi potrzebami wizualnymi.
- Wszystkie kombinacje tekst/tło mają wystarczający kontrast kolorów, zgodny z wymaganiami WCAG AA (minimum 4.5:1 dla zwykłego tekstu).
- Wiele elementów posiada teksty alternatywne, np. opisujące stan połączenia z brokerem na liście urządzeń, co umożliwia odczyt informacji przez czytniki ekranu.

- Pojawiają się problemy z przyciskami ikonami gdzie brakuje tekstu alternatywnego - jest to problem użytej biblioteki do komponentów które nie udostępnia możliwości nadpisania tekstu alternatywnego.

2. Obsługiwanie (Operable)

- Interfejs jest w pełni dostępny przy użyciu klawiatury, umożliwiając użytkowanie aplikacji osobom, które nie mogą korzystać z myszy.
- W przypadku treści wykraczających poza ekran wyświetlane są scrollbary, pozwalające użytkownikowi dotrzeć do wszystkich elementów interfejsu.

3. Zrozumiałe (Understandable)

- Aplikacja wyświetla powiadomienia (snack bary) informujące o błędach lub ważnych zdarzeniach.
- Ikony są przewidywalne i spójne, a układ stron ujednolicony: każdy ekran zawiera FAB (Floating Action Button) z opcjami specyficznymi dla danej strony.
- Breadcrumby wspierają intuicyjną nawigację i łatwe zorientowanie się w strukturze aplikacji.
- Błędy w formularzach są wyraźnie oznaczone kolorem czerwonym i umieszczone bezpośrednio pod problematycznym polem, co umożliwia szybkie ich zidentyfikowanie i poprawę.

4. Solidne (Robust)

- Interfejs jest responsywny i automatycznie dopasowuje się do różnych rozmiarów ekranów: telefony, tablety, desktopy.
- Tekst można skalować do 200% bez utraty funkcjonalności lub czytelności interfejsu.
- Kompatybilność z różnymi przeglądarkami i urządzeniami zapewnia stabilne działanie aplikacji.

Wtyczka WAVE ocenia rozwiązanie na ~9 w skali 1-10 z drobnymi skokami w górę oraz w dół.

Walidacja Danych

Każdy formularz używa modelu danych który zawiera określone właściwości pola. Jeżeli wartość jest niezgodna nie da się wysłać zapytania o dodanie elementu a problematyczny element jest podkreślony czerwonym kolorem wraz z opisem problemu (Rys. 8). W wypadku problemu z zapytaniem wyświetlany jest snackbar z informacja o błędzie

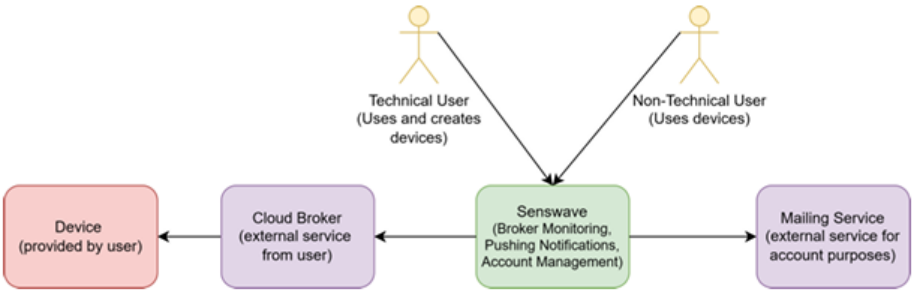
Rys.8 Formularz tworzenia urządzenia z zaznaczonym błędem.

Backend

Backend został w całości napisany w technolog .NET ASP Core z wykorzystaniem Bazy danych Postgres oraz RabbitMq jako rozwiązania przesyłającego komunikaty..

Kod tej części systemu nie został udostępniony jako że nie był realizowany w ramach tego zadania jeżeli jest konieczność udostępnienia tej czesci prosze o kontakt.

Kontekst Systemu



Rys.9 Kontekst działania systemu.

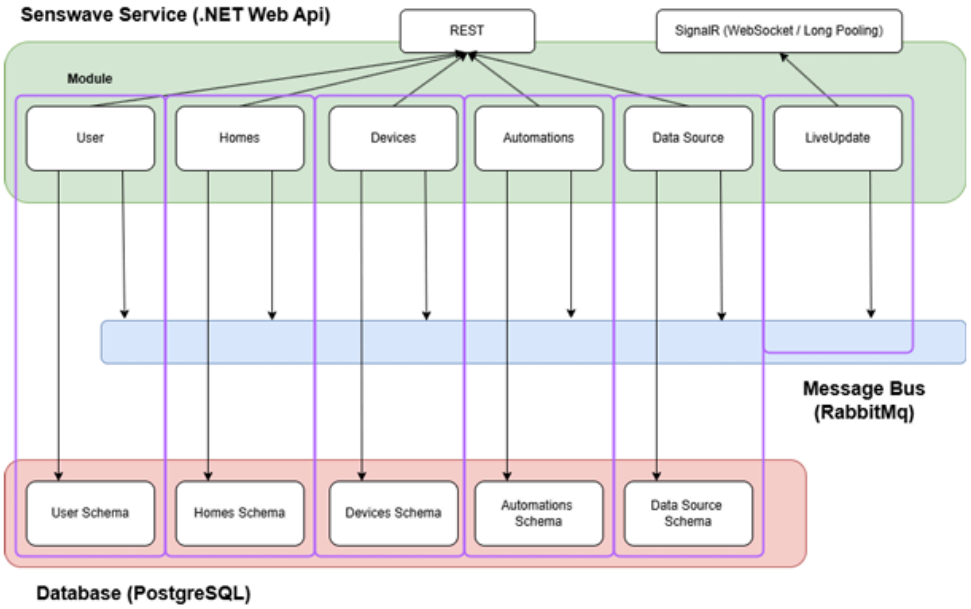
System Senswave został zaprojektowany jako platforma pośrednicząca w komunikacji między urządzeniami IoT a użytkownikami końcowymi (Rys. 9). Jego poprawne funkcjonowanie opiera się na integracji z kluczowymi komponentami zewnętrznymi:

- **Platformy chmurowe i brokerzy MQTT:** Wykorzystanie brokera HiveMQ stanowi fundament komunikacyjny, umożliwiający dwukierunkową wymianę danych między warstwą sprzętową a logiczną systemu.
- **Infrastruktura pocztowa:** Zewnętrzny serwer mailowy integruje procesy uwierzytelniania, wspierając zarządzanie kontami i bezpieczeństwo dostępu.

System adresuje potrzeby dwóch grup odbiorców: użytkowników technicznych, operujących na poziomie konfiguracji i rejestracji urządzeń, oraz użytkowników nietechnicznych, których interakcja ogranicza się do konsumpcji funkcji dostarczanych przez gotowe urządzenia.

Architektura wewnętrzna - Kontenery

Architektura rozwiązania została oparta na paradygmacie **modularnego monolitu** (Rys. 10), co pozwala na zachowanie spójności kodu przy jednoczesnym logicznym odseparowaniu odpowiedzialności. Całość infrastruktury została zorientowana na konteneryzację przy użyciu technologii **Docker**, co gwarantuje powtarzalność środowisk deweloperskich i produkcyjnych.



Rys.10 Architektura systemu backendowego.

Komunikacja i Bezpieczeństwo

Mechanizmy zabezpieczeń

Bezpieczeństwo systemu opiera się na trzech filarach: autoryzacji, szyfrowaniu oraz kontroli dostępu.

- **System Tokenów (Bearer Tokens):** Dostęp do funkcji systemu jest chroniony za pomocą pary tokenów. **Access token** pozwala na korzystanie z API przez godzinę, natomiast **refresh token**, ważny przez 90 dni, umożliwia automatyczne przedłużenie sesji bez konieczności ponownego logowania.
- **Szyfrowanie HTTPS i WSS:** Cała komunikacja między aplikacją mobilną a serwerem w środowisku chmurowym jest szyfrowana. Uniemożliwia to przejęcie danych przez osoby trzecie podczas ich przesyłania.
- **Ograniczenie dostępu (CORS):** Serwer akceptuje żądania wyłącznie z zaufanych źródeł zdefiniowanych w konfiguracji systemu. Dodatkowo, dzięki przesyłaniu tokenów w nagłówkach, system jest odporny na ataki typu CSRF.

Interfejs REST API

Standardowa komunikacja z serwerem odbywa się poprzez endpointy REST. Są one podzielone na dwie grupy:

- **Otwarte:** dostępne dla każdego (np. ekran logowania czy rejestracji).
- **Chronione:** wymagające przesłania ważnego tokenu w nagłówku zapytania. To tutaj odbywa się cała kluczowa logika zarządzania urządzeniami.

Łączność w czasie rzeczywistym (SignalR)

Aby zapewnić natychmiastowe powiadomienia o stanie urządzeń, system wykorzystuje technologię **SignalR**. Jest to zaawansowane rozwiązanie, które automatycznie dobiera najlepszy sposób komunikacji:

1. **WebSockets:** Podstawowa i najszybsza metoda pozwalająca na dwukierunkową wymianę danych.
2. **Mechanizmy zapasowe:** Jeśli urządzenie użytkownika lub sieć nie obsługują standardu WebSockets, SignalR automatycznie przełącza się na prostsze metody (Server-Sent Events lub Long Polling).

Dzięki temu użytkownik widzi aktualizacje bez konieczności ręcznego odświeżania ekranu aplikacji. Podobnie jak w przypadku REST API, to połączenie jest w pełni autoryzowane i szyfrowane (WSS).